



PSOC™ 4 CAPSENSE™ built-in self-test

Customer training workshop (CTW)

August 2026



Prerequisites

- Basic hands-on with ModusToolbox™
- Basic understanding of PSOC™ 4xxx device architecture
- [PSOC™ 4 CAPSENSE™ using PSOC™ 4000T and PSOC™ 4100T Plus training](#)
- Install or update the following on your PC:
 - ModusToolbox™ Tools Package v3.7.0
 - Python v3.12.x (Python v3.8.x to v3.12.x are supported by ModusToolbox™ v3.7.0)
 - ModusToolbox™ Eclipse IDE 2026.1.0
 - ModusToolbox™ Programming Tools 1.4
 - ModusToolbox™ CAPSENSE™ and Multi-sense Pack v1.3.0

Agenda

1	Introduction to the CAPSENSE™ middleware self-test library	4
2	Description of the CAPSENSE™ self-tests available	8
3	Lab 1: Executing the built-in self-tests	19
4	Lab 2: Executing the built-in self-tests periodically	21

Table of contents

1	Introduction to the CAPSENSE™ middleware self-test library	4
2	Description of the CAPSENSE™ self-tests available	8
3	Lab 1: Executing the built-in self-tests	19
4	Lab 2: Executing the built-in self-tests periodically	21

Built-in self-tests

- APIs built into the CAPSENSE™ middleware
- To verify that the CAPSENSE™ block is functional
 - Verify that the RAM used to maintain CAPSENSE™ widget data is valid
 - Verify the sensor integrity
 - Verify the shield integrity
 - Verify the analog voltage reference integrity
 - Verify the integrity of the external capacitors

Run built-in self-tests

- Manufacturing tests
- Device initialization
- Runtime

Measured capacitance calculation by middleware

- Many of the tests in the middleware measure the capacitance of either sensors or the integration capacitors
- For the capacitance measurement, the CSD method is used
- The following formula is used to calculate the capacitance:
 - $C_s = \text{Rawcount} * C_{\text{ref}} / \text{NSub}$
 - C_s is the sensor capacitance
 - Rawcount is the measured raw count value
 - C_{ref} (887fF) is the reference CDAC value used during the test
 - NSub (100) is the number of sub-conversions used during the test

Agenda

1	Introduction to the CAPSENSE™ middleware self-test library	4
2	Description of the CAPSENSE™ self-tests available	8
3	Lab 1: Executing the built-in self-tests	19
4	Lab 2: Executing the built-in self-tests periodically	21

CAPSENSE™ middleware structures used for BIST

- **cy_stc_capsense_context_t**: Top-level CAPSENSE™ context data structure
 - Contains pointers to other CAPSENSE™ middleware data structures
- **cy_stc_capsense_bist_context_t**: BIST context data structure, used to hold information such as:
 - The hardware configuration for BIST
 - The current state of the sensors, electrodes, and shields
 - Test results
- **cy_stc_capsense_widget_config_t**: Widget configuration data structure, used to hold information such as:
 - Pointers to widget and sensor context information
 - Pointers to electrode and sensor capacitance calculated values
 - Other widget configurations

High-level self-test API

- High-level function that performs various self-tests on all enabled widgets and sensors in the project
- The required set of tests can be selected using the bitmask in the `testEnMask` parameter
 - `CY_CAPSENSE_BIST_CRC_WDGT_MASK`
 - `CY_CAPSENSE_BIST_SNS_INTEGRITY_MASK`
 - `CY_CAPSENSE_BIST_SNS_CAP_MASK`
 - `CY_CAPSENSE_BIST_ELTD_CAP_MASK`
 - `CY_CAPSENSE_BIST_SHIELD_CAP_MASK`
 - `CY_CAPSENSE_BIST_EXTERNAL_CAP_MASK`
 - `CY_CAPSENSE_BIST_VDDA_MASK`
 - `CY_CAPSENSE_BIST_RUN_AVAILABLE_SELF_TEST_MASK`

```
cy_en_capsense_bist_status_t Cy_CapSense_RunSelfTest(uint32_t testEnMask,  
cy_stc_capsense_context_t * context)
```

External capacitance measurements

- Measures the capacitance of the specified CAPSENSE™ integration capacitor
- The maximum that can be measured is 25 nF
- The measurement accuracy is 30% for the fifth-generation MSCLP CAPSENSE™
- The nominal value recommended is 2.2 nF
 - The maximum capacitance parameter should be set to 4 nF for this API

```
cy_en_capsense_bist_status_t  
Cy_CapSense_MeasureCapacitanceCap(cy_en_capsense_bist_external_cap_id_t integrationCapId,  
uint32_t* ptrValue,uint32_t maxCapacitance,cy_stc_capsense_context_t* context)
```

Electrode capacitance measurement

- Measures the sensor capacitance for CSD widgets or the electrode capacitance for CSX widgets
- This API will measure the ganged sensor capacitance for ganged sensor widgets

```
cy_en_capsense_bist_status_t Cy_CapSense_MeasureCapacitanceSensorElectrode  
(uint32_t widgetId, uint32_t eltdId, cy_stc_capsense_context_t* context5)
```

Sensor slot capacitance measurement

- Measures the self capacitance for CSD widgets or the mutual capacitance for CSX widgets

```
cy_en_capsense_bist_status_t Cy_CapSense_MeasureCapacitanceSlotSensors(uint32_t  
slotId, uint32_t skipChMask, cy_stc_capsense_context_t* context
```

Shield electrode measurement

- Measures the capacitance of all shield electrodes
- This API checks if there are any CSD widgets in the project and if the shield is enabled
- If a shield channel consists of multiple electrodes, the total capacitance of all the shield electrodes is measured

```
cy_en_capsense_bist_status_t Cy_CapSense_MeasureCapacitanceShieldElectrode(uint32_t  
skipChMask, cy_stc_capsense_context_t* context)
```

CRC of the parameter set

- Verifies parameter integrity during tests or after parameter updates
- Uses the CRC of widget parameters to verify that the data is not corrupt

```
cy_en_capsense_bist_status_t Cy_CapSense_CheckCRCWidget(uint32_t widgetId,  
cy_stc_capsense_context_t* context)
```

Rawcount measurement

- Validates that the raw count falls within the limits specified by the function parameters
- This function must be called after sensor scanning and before any processing is performed

```
cy_en_capsense_bist_status_t Cy_CapSense_CheckIntegritySensorRawcount(uint32_t widgetId,  
uint32_t sensorId, uint16_t rawcountHighLimit, uint16_t rawcountLowLimit,  
cy_stc_capsense_context_t* context)
```

Pin open and short tests

- Checks that the specified sensor is not electrically shorted
- Detects shorts for the following:
 - GND
 - VDD
 - Other GPIOs used by CAPSENSE™
 - Other non-CAPSENSE™ GPIOs

```
cy_en_capsense_bist_status_t Cy_CapSense_CheckIntegritySensorPins(uint32_t widgetId,  
uint32_t sensorId, cy_stc_capsense_context_t* context)
```

VDDA measurement

- Measures the analog supply, VDDA, internally
- Useful in battery powered applications to verify the VDDA voltage is above the required value

```
cy_stc_capsense_bist_context_t Cy_CapSense_MeasureVdda(uint32_t* ptrValue,  
cy_stc_capsense_context_t* context)
```

Agenda

1	Introduction to the CAPSENSE™ middleware self-test library	4
2	Description of the CAPSENSE™ self-tests available	8
3	Lab 1: Executing the built-in self-tests	19
4	Lab 2: Executing the built-in self-tests periodically	21

Lab 1: Executing the built-in self- tests

What will you learn ?

- How to enable the built-in self-tests
- How to execute them in the source code
- How to get the results

Agenda

1	Introduction to the CAPSENSE™ middleware self-test library	4
2	Description of the CAPSENSE™ self-tests available	8
3	Lab 1: Executing the built-in self-tests	19
4	Lab 2: Executing the built-in self-tests periodically	21

Lab 2: Executing the built-in self-tests periodically during runtime

What will you learn?

- How to modify the application to enable runtime self-tests

Revision history

Document revision	Date	Description of change
**	2026-08-28	Initial release.

